



FRIEDRICH-ALEXANDER-UNIVERSITÄT
ERLANGEN-NÜRNBERG (FAU)

NHR@FAU

Assignment 2: More code execution, loop benchmarking

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

May 22, 2025

Q1: Theoretical Peak Performance

To calculate the theoretical double-precision floating-point peak performance of the CPU, we use the following specifications:

- Clock frequency: 2.0 GHz
- Number of cores: 52
- Instruction set: AVX-512 (512-bit SIMD)
- Each core can retire 2 double-precision FMA instructions per cycle

Each AVX-512 register can hold 8 double-precision values:

$$\frac{512 \text{ bits}}{64 \text{ bits}} = 8 \text{ doubles}$$

Each FMA (fused multiply-add) operation counts as 2 floating-point operations (1 multiplication and 1 addition). So, for each cycle per core:

$$\text{Flops/cycle/core} = 2 \text{ FMA} \times 8 \text{ doubles} \times 2 \text{ flops/FMA} = 32 \text{ flops}$$

Total peak performance of the chip:

$$\text{Peak} = 32 \text{ flops/cycle/core} \times 2.0 \times 10^9 \text{ cycles/s} \times 52 \text{ cores} = 3.328 \times 10^{12} \text{ flops/s}$$

$$\boxed{\text{Peak Performance} = 3.328 \text{ T}}$$

This is the theoretical peak assuming fully parallel SIMD execution with no stalls or overheads.

Q2: Pipelining and Performance Bottlenecks

We analyze a loop kernel with a scalar accumulation and a scaling operation:

```
for (int i = 0; i < N; ++i) {
    s += a[i] * a[i];
    b[i] *= t;
}
```

(a) Compiler without Modulo Variable Expansion (MVE)

- ADD pipeline latency: 5 cycles
- Cross-iteration dependency on scalar **s**
- FLOPs per iteration: 2 (MULT) + 1 (ADD) = 3 FLOPs

Due to the loop-carried dependency on **s**, each iteration must wait for the previous ADD to finish. Hence, a new iteration can begin only every 5 cycles.

$$P_{\text{no MVE}} = \frac{3 \text{ FLOPs}}{5 \text{ cy}} = \boxed{0.60 \text{ FLOPs/cy}}$$

All other hardware limits (load/store bandwidth, instruction retirement) remain underutilized. The bottleneck is the dependency on the ADD instruction.

(b) Compiler with perfect MVE

To hide the 5-cycle ADD latency, we apply MVE using at least:

$$m = \left\lceil \frac{5 \text{ cy}}{2 \text{ cy}} \right\rceil = 3 \text{ independent accumulators}$$

After MVE, the iteration becomes limited by the load bandwidth. Each iteration performs:

- 2 LOADs + 1 STORE
- 2 MULTs
- 1 ADD

Since only 1 LOAD per cycle is possible, a new iteration can start every 2 cycles:

$$P_{\text{best}} = \frac{3 \text{ FLOPs}}{2 \text{ cy}} = \boxed{1.5 \text{ FLOPs/cy}}$$

This saturates the load unit while keeping all other execution units within their limits.

Q3: Loop Kernel Benchmarking

Objective

The goal was to benchmark the performance of two loop kernels:

1. **Schönauer triad:** `a[i] = b[i] + c[i] * d[i];`
2. **DAXPY:** `a[i] = s * b[i] + a[i];`

where `s = 1.00000000001` is a compile-time constant, and all arrays are double-precision floating-point vectors of length `N`.

Implementation

The program was written in C using heap-allocated memory with proper initialization of all arrays to avoid undefined behavior. Performance was measured in MFlop/s using wall-clock timing with adaptive iteration counts to ensure at least 0.1s of kernel runtime per vector length. The compiler was configured with the following flags to enable aggressive optimization and disable aliasing assumptions:

```
icc -O3 -xHost -fno-alias -o benchmark benchmark.c
```

Benchmark Execution

The benchmark was executed on a single core of the Fritz cluster with fixed clock frequency using:

```
salloc -nodes=1 -time=01:00:00
module load intel
srun -cpu-freq=2200000-2200000:performance ./benchmark > results.csv
```

where the `srun` command fixes the CPU frequency at 2.2 GHz to ensure consistent and reproducible performance measurements.

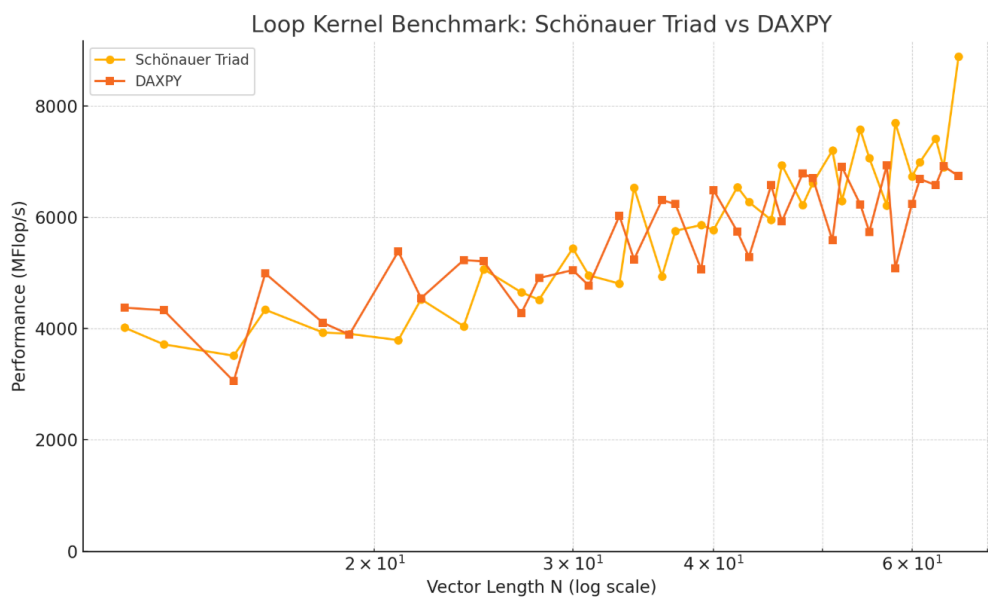
Benchmark Conditions

- Single core execution on Fritz cluster
- Fixed clock frequency: 2.2 GHz
- Vector lengths: $N = \text{int}(1.5 \times r)$ for $r = 8, \dots, 44$
- Output: Performance measured in MFlop/s as a function of vector length N

Results and Discussion

The results show the expected performance trends:

- Both kernels exhibit increasing performance with increasing N , stabilizing at larger sizes due to cache and memory effects.
- The Schönauer triad consistently achieves slightly higher peak performance than DAXPY, likely because of more regular memory access and instruction patterns.
- Performance curves reflect memory hierarchy behavior: low throughput at small vector lengths due to cache warm-up and loop overhead, followed by a rapid increase and eventual plateau caused by memory bandwidth saturation.



The logarithmic x-axis allows clear visualization of performance growth from small to large problem sizes, while the y-axis starting at zero prevents misleading impressions. The observed results align well with theoretical expectations and the discussions in lectures.

Appendix: Benchmark Code (C)

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <math.h>

#define S 1.000000000001

double getTimeStamp() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (double)tv.tv_sec + 1e-6 * tv.tv_usec;
}

void initialize_arrays(double *a, double *b, double *c, double *d, int N) {
    for (int i = 0; i < N; ++i) {
        a[i] = 0.0;
        b[i] = 1.0 + i * 0.001;
        c[i] = 2.0 + i * 0.002;
        d[i] = 3.0 + i * 0.003;
    }
}

void run_kernel(const char *label, int kernel_type) {
    printf("\n%s Performance (MFlop/s):\n", label);
    printf("N, MFlop/s\n");

    for (int r = 8; r <= 44; ++r) {
        int N = (int)(1.5 * r);
        double *a = (double *)malloc(N * sizeof(double));
        double *b = (double *)malloc(N * sizeof(double));
        double *c = (double *)malloc(N * sizeof(double));
        double *d = (double *)malloc(N * sizeof(double));

        if (!a || !b || !c || !d) {
            printf("Memory allocation failed!\n");
            exit(EXIT_FAILURE);
        }

        initialize_arrays(a, b, c, d, N);

        int NITER = 1;
        double start, end, runtime;

        do {
            start = getTimeStamp();
            for (int k = 0; k < NITER; ++k) {
                for (int i = 0; i < N; ++i) {
                    if (kernel_type == 0) {
                        a[i] = b[i] + c[i] * d[i]; // Schnauer Triad
                    } else {
                        a[i] = S * b[i] + a[i]; // DAXPY
                    }
                }
                if (a[N / 2] < 0.) printf("%lf\n", a[N / 2]); // prevent elimination
            }
            end = getTimeStamp();

```

```
        runtime = end - start;
        if (runtime < 0.1) NITER *= 2;
    } while (runtime < 0.1);

    int flops_per_iter = 2; // 1 MUL + 1 ADD
    double mflops = (double)(NITER * N * flops_per_iter) / (runtime * 1.0e6);
    printf("%d, %.2f\n", N, mflops);

    free(a); free(b); free(c); free(d);
}

int main() {
    run_kernel("Schnauer Triad", 0);
    run_kernel("DAXPY", 1);
    return 0;
}
```

Listing 1: Benchmarking Schönauer Triad and DAXPY Kernels